Lab 7: Fuzzing for Input Validation Bugs

SRN: PES1UG24CS058 **Student:** Anagha
Assigned repo: L6_L7_01_water_utility_billing (Water Utility Billing)
Tool: Hypothesis (property-based fuzzing) with pytest, Python 3.14.3

Summary

The three functions in `processor.py` were fuzzed with Hypothesis using `st.text() | st.none()`. All three crashed on edge-case input. They were hardened with defensive checks and all fuzz tests now pass.

Function	Bugs found by fuzzing	Fix
sanitize_meter_id	None input: AttributeError ('NoneType' has no attribute 'strip')	Return "" for non-string input
parse_consumption_readings	None: AttributeError. Empty / non-numeric element (e.g. '', 'abc', ' '): ValueError	Return [] for non-string; skip elements that are not integers
format_billing_summary	None: AttributeError. Missing fields ('' or 'a'): IndexError. Non-integer usage (':' or 'a:b:c'): ValueError	Return 'Invalid account record' for non-string, wrong field count, empty name/zone, non-integer or negative usage

Step 1: Install dependencies

`pip install hypothesis` and `pip install pytest` (hypothesis 6.170.15, pytest 9.1.1).

Step 2: Run the buggy application (`python processor.py`)

Run 1 uses a readings list with an empty element; run 2 uses a non-numeric usage value. Each crashes with an unhandled exception.

Lab7 Buggy Run 1

```
PS> python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): #m-42$
Sanitized Meter ID: M-42

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): 10,25,,abc,30
Traceback (most recent call last):
  File "C:\Users\Anagha\AppData\Local\Temp\claude\5eb326e563be04f2\a7962e0f-067e-4099-aa0b-
84853cbd3fbd\scratchpad\buggy\processor.py", line 35, in <module>
    main()
    ~~~~^^
  File "C:\Users\Anagha\AppData\Local\Temp\claude\5eb326e563be04f2\a7962e0f-067e-4099-aa0b-
84853cbd3fbd\scratchpad\buggy\processor.py", line 26, in main
    readings_list = parse_consumption_readings(csv_input)
  File "C:\Users\Anagha\AppData\Local\Temp\claude\5eb326e563be04f2\a7962e0f-067e-4099-aa0b-
84853cbd3fbd\scratchpad\buggy\processor.py", line 9, in parse_consumption_readings
    return [int(r) for r in readings]
            ~~~^^^
ValueError: invalid literal for int() with base 10: ''
```

Buggy processor.py: ValueError on input '10,25,,abc,30'

Lab7 Buggy Run 2

```
PS> python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): !@#wm-1001%
Sanitized Meter ID: WM-1001

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): 10,25,30
Parsed Readings: [10, 25, 30]

==== Running format_billing_summary ====
Enter account record (Name:Usage:Zone): Ravi:abc:North
Traceback (most recent call last):
  File "C:\Users\Anagha\AppData\Local\Temp\claude\5eb326e563be04f2\a7962e0f-067e-4099-aa0b-
84853cbd3fbd\scratchpad\buggy\processor.py", line 35, in <module>
    main()
    ~~~~^^
  File "C:\Users\Anagha\AppData\Local\Temp\claude\5eb326e563be04f2\a7962e0f-067e-4099-aa0b-
84853cbd3fbd\scratchpad\buggy\processor.py", line 31, in main
    summary = format_billing_summary(summary_input)
  File "C:\Users\Anagha\AppData\Local\Temp\claude\5eb326e563be04f2\a7962e0f-067e-4099-aa0b-
84853cbd3fbd\scratchpad\buggy\processor.py", line 14, in format_billing_summary
    units = int(parts[1])
ValueError: invalid literal for int() with base 10: 'abc'
```

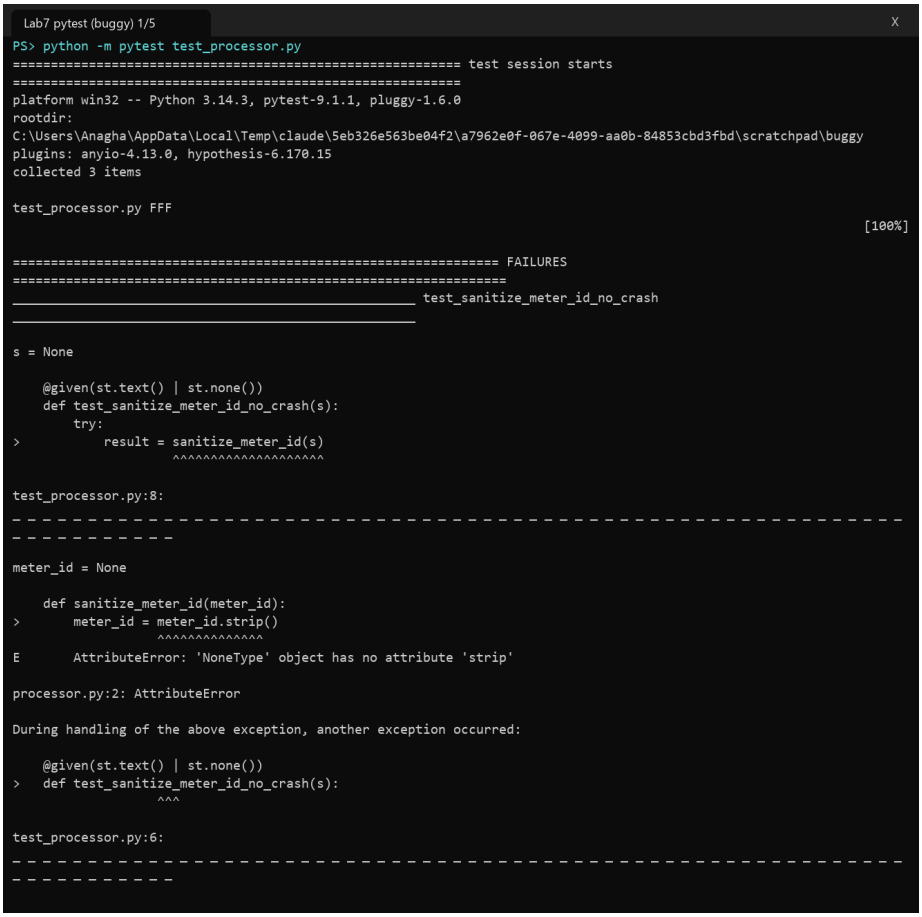
Buggy processor.py: ValueError on account record 'Ravi:abc:North'

Step 3: Complete `test_processor.py` using Hypothesis

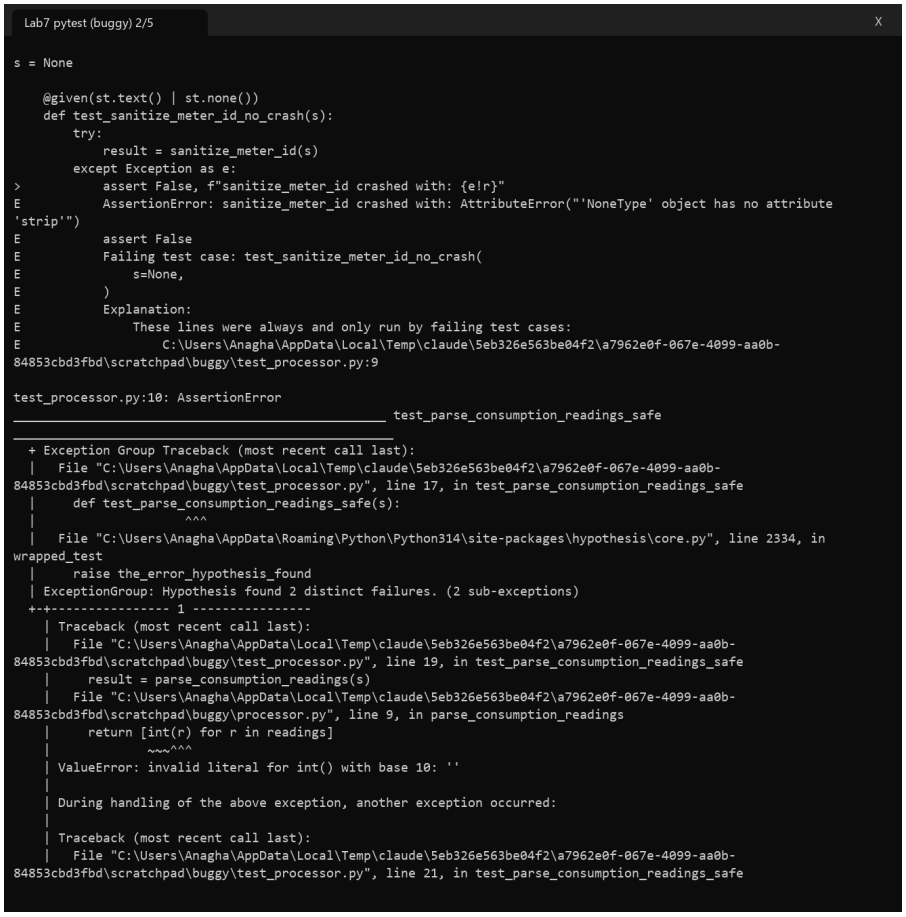
Each test wraps the call in try/except and fails with the exception caught, then also asserts the result type (str / list of int) and, for sanitize_meter_id, that no prefix symbols remain and the result is upper-case. The full code is in the appendix.

Step 4: Run `pytest test_processor.py` against the buggy code

All 3 tests fail. Hypothesis reports minimal failing inputs: None, '' and ':' among others (output split over several screenshots).



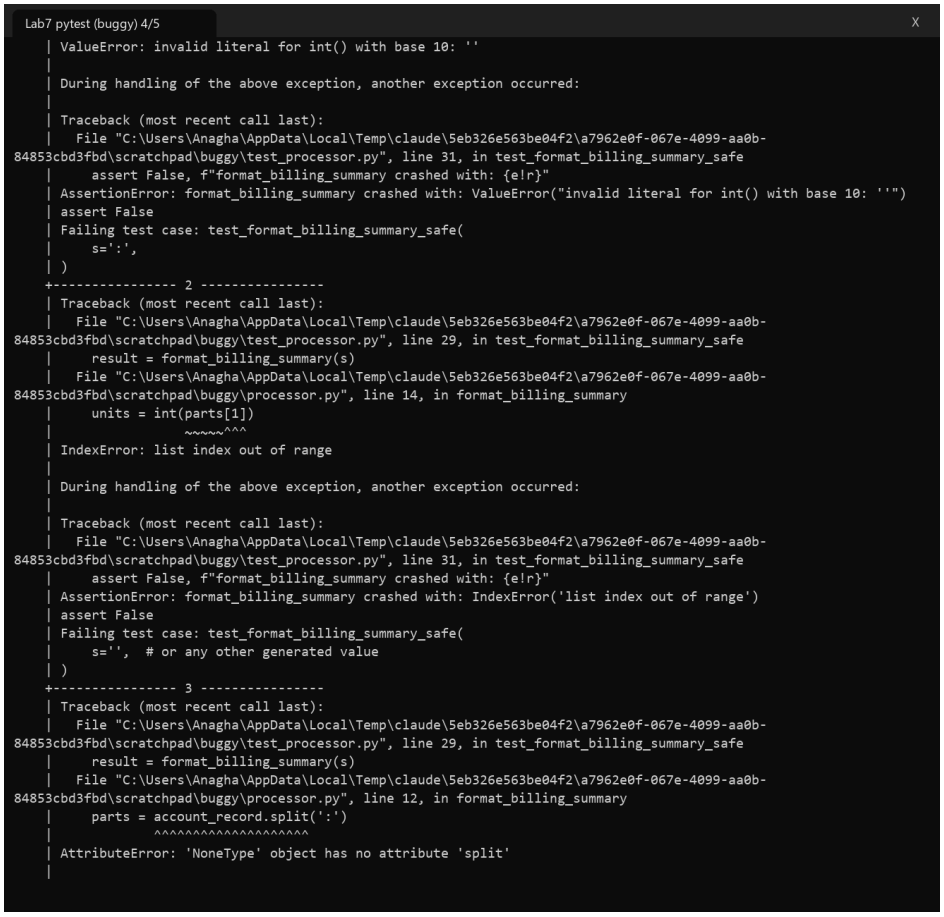
step4 pytest failing part1of5



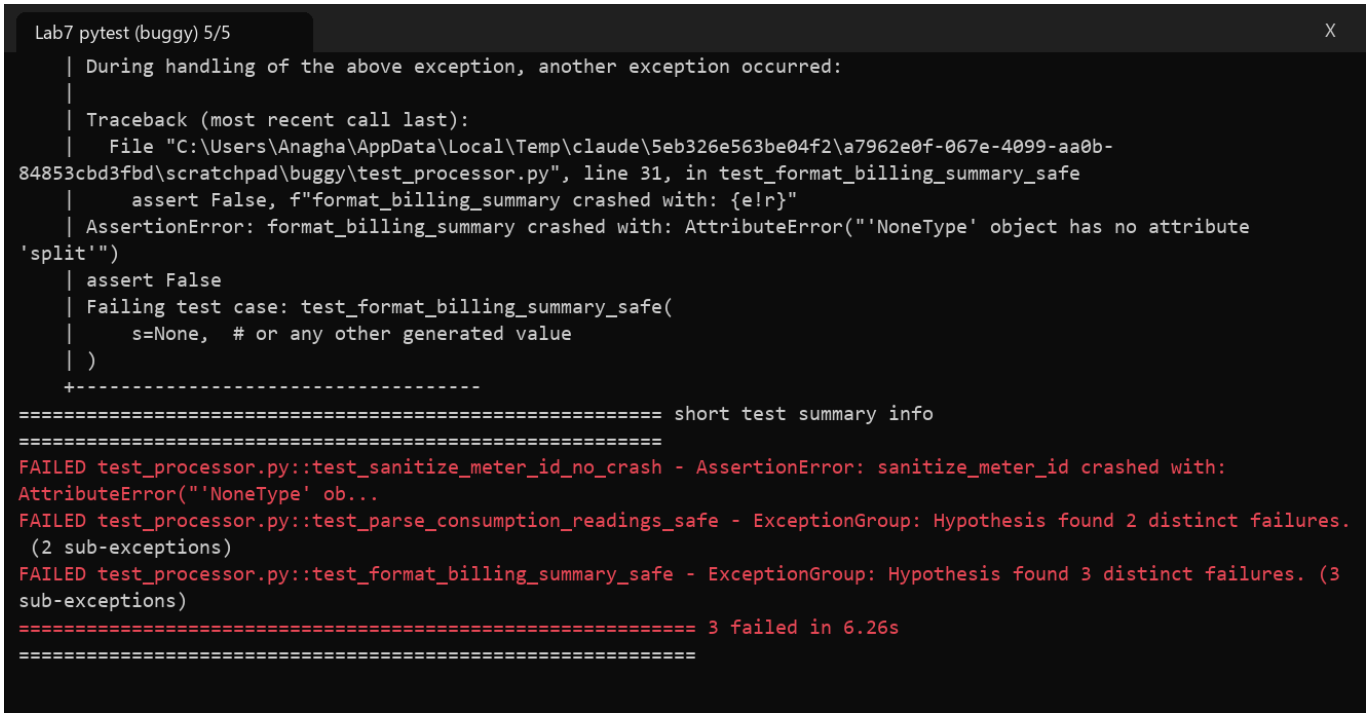
step4 pytest failing part2of5



step4 pytest failing part3of5



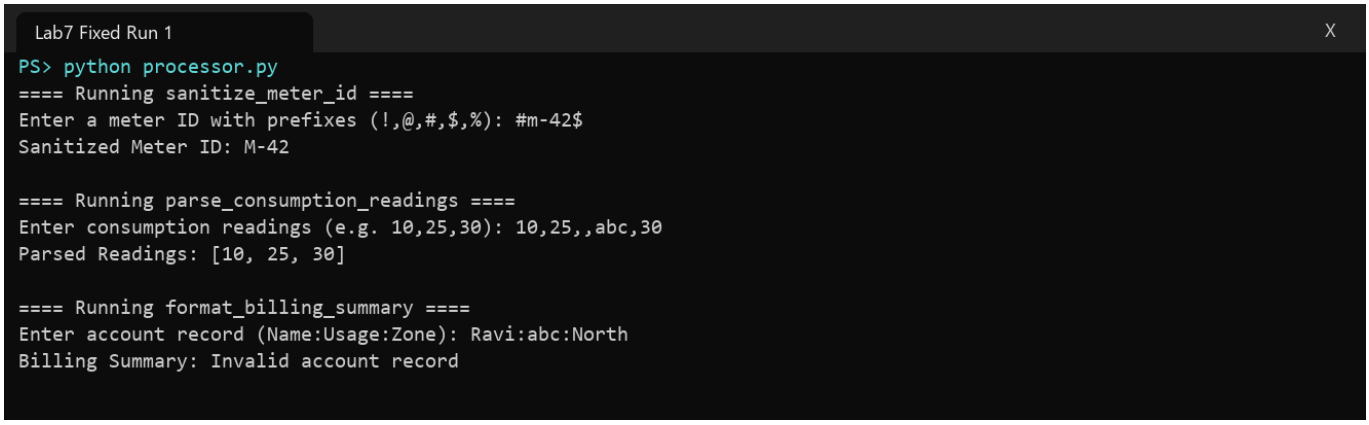
step4 pytest failing part4of5



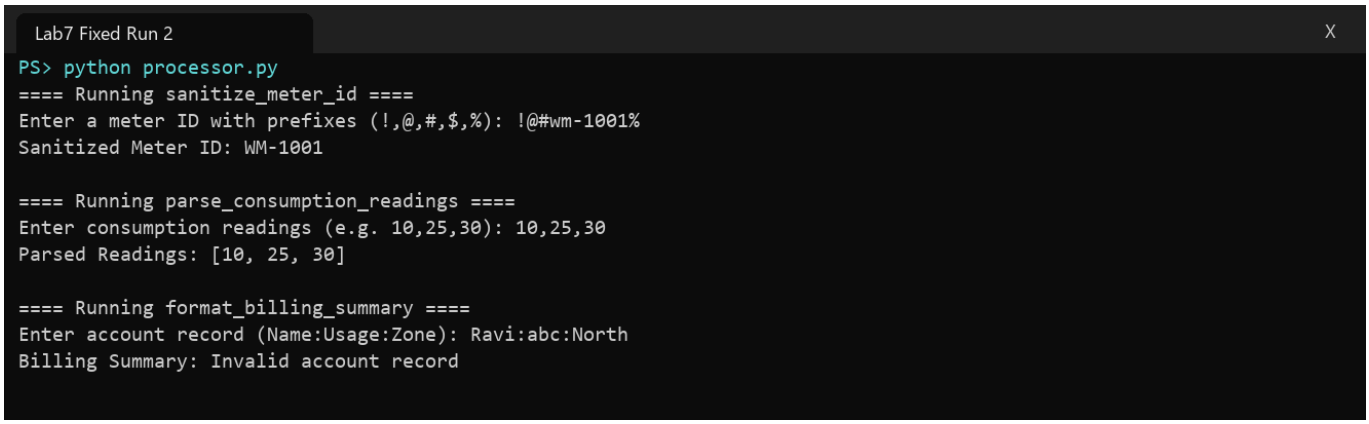
step4 pytest failing part5of5

Step 5: Fix `processor.py` and re-run it

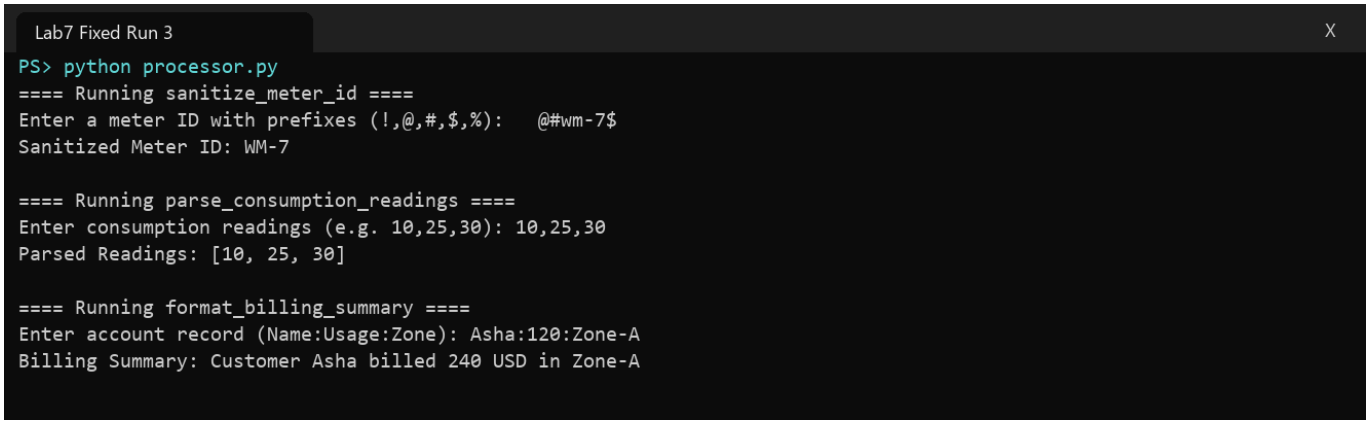
Same inputs as the buggy runs now produce clean output instead of tracebacks; run 3 shows normal valid input still works.



Fixed processor.py: same input as buggy run 1 (invalid readings skipped)

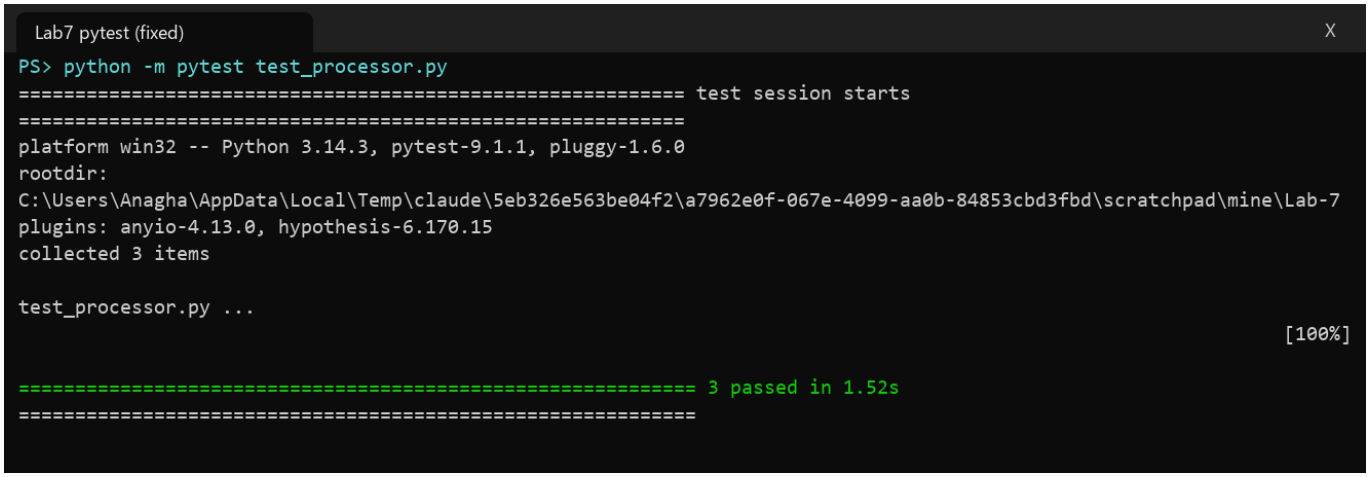


Fixed processor.py: same input as buggy run 2 (invalid record handled)



Fixed processor.py: valid input still produces the correct summary

Step 6: Re-run `pytest test_processor.py` against the fixed code



All 3 fuzz tests pass

Appendix A: Fixed processor.py

```
INVALID_RECORD_MSG = "Invalid account record"


def sanitize_meter_id(meter_id):
    """Strip whitespace and the prefix symbols !@#$%, then upper-case.

    Non-string input (e.g. None) yields an empty string instead of crashing.
    """
    if not isinstance(meter_id, str):
        return ""
    meter_id = meter_id.strip()
    for ch in ['!', '@', '#', '$', '%']:
        meter_id = meter_id.replace(ch, '')
    return meter_id.upper()


def parse_consumption_readings(readings_csv):
    """Parse a comma-separated string of integers.

    Empty, malformed or non-numeric elements are skipped; non-string input
    yields an empty list.
    """
    if not isinstance(readings_csv, str):
        return []
    readings = []
    for item in readings_csv.split(','):
        try:
            readings.append(int(item.strip()))
        except ValueError:
            continue
    return readings


def format_billing_summary(account_record):
    """Format a 'Name:Usage:Zone' record as a billing summary.

    Returns INVALID_RECORD_MSG for non-string input, a wrong number of
    fields, an empty name/zone, or non-integer / negative usage.
    """
    if not isinstance(account_record, str):
        return INVALID_RECORD_MSG
    parts = account_record.split(':')
    if len(parts) != 3:
        return INVALID_RECORD_MSG
    customer_name = parts[0].strip()
    zone = parts[2].strip()
    if not customer_name or not zone:
        return INVALID_RECORD_MSG
    try:
        units = int(parts[1].strip())
    except ValueError:
        return INVALID_RECORD_MSG
    if units < 0:
        return INVALID_RECORD_MSG
    return f"Customer {customer_name} billed {units * 2} USD in {zone}"


def main():
    print("==== Running sanitize_meter_id ====")
    raw_id = input("Enter a meter ID with prefixes (!,@,#,$,%): ")
    clean_id = sanitize_meter_id(raw_id)
    print("Sanitized Meter ID:", clean_id)

    print("\n==== Running parse_consumption_readings ====")
    csv_input = input("Enter consumption readings (e.g. 10,25,30): ")
    readings_list = parse_consumption_readings(csv_input)
    print("Parsed Readings:", readings_list)

    print("\n==== Running format_billing_summary ====")
    summary_input = input("Enter account record (Name:Usage:Zone): ")
    summary = format_billing_summary(summary_input)
    print("Billing Summary:", summary)

if __name__ == "__main__":
    main()
```

Appendix B: Updated test_processor.py

```
from hypothesis import given, strategies as st
from processor import sanitize_meter_id, parse_consumption_readings, format_billing_summary


@given(st.text() | st.none())
def test_sanitize_meter_id_no_crash(s):
    try:
        result = sanitize_meter_id(s)
    except Exception as e:
        assert False, f"sanitize_meter_id crashed with: {e!r}"
    assert isinstance(result, str)
    assert not any(ch in result for ch in "!@#$%")
    assert result == result.upper()


@given(st.text() | st.none())
def test_parse_consumption_readings_safe(s):
    try:
        result = parse_consumption_readings(s)
    except Exception as e:
        assert False, f"parse_consumption_readings crashed with: {e!r}"
    assert isinstance(result, list)
    assert all(isinstance(r, int) for r in result)


@given(st.text() | st.none())
def test_format_billing_summary_safe(s):
    try:
        result = format_billing_summary(s)
    except Exception as e:
        assert False, f"format_billing_summary crashed with: {e!r}"
    assert isinstance(result, str)
```